Introduction to Cryptography

Lecture 16: Key Management and Distribution

Dr. Siddhant Thapliyal

Department of Computer Science and Engineering
Graphic Era (Deemed to be University), Dehradun

Why Do We Need Key Distribution?

Basic Requirement

For symmetric encryption to work, both communicating parties must possess the **same secret key**.

This creates an important question:

How can the secret key be delivered securely?

- The key must remain secret from attackers.
- Keys should often be changed periodically.
- A compromised key should not expose excessive amounts of data.

What is Key Distribution?

Definition

Key distribution refers to the process of delivering a cryptographic key to communicating parties without allowing unauthorized entities to obtain it.

If Alice and Bob want to communicate securely:

$$Alice \longleftrightarrow Bob$$

both must somehow obtain the same secret key:

$$K_{AB}$$

without an attacker learning it.

Ways to Distribute a Symmetric Key

For two communicating parties A and B, a key can be distributed in several ways:

1. A selects the key and physically delivers it to B.
2. A trusted third party selects and physically delivers the key to both.
3. A new key is encrypted using an already shared old key.
4. A trusted third party delivers a key through encrypted connections.

Observation

Manual distribution becomes difficult when a network contains a large number of users or devices.

Manual Key Distribution

Advantages

- Simple concept
- Can be secure in small systems
- Useful for dedicated links

Problems

- Difficult to scale
- Requires physical interaction
- Expensive for large networks
- Frequent key changes become impractical

The Key Distribution Scalability Problem

Suppose a network contains N users.
If every pair requires a unique symmetric key:

$$\boxed{\text{Number of Keys} = \frac{N(N-1)}{2}}$$

This grows rapidly as N increases.

Numerical Example: Number of Keys

Suppose there are:

$$N = 1000$$

users.
The number of possible communicating pairs is:

$$\frac{N(N-1)}{2}$$

$$= \frac{1000(999)}{2}$$

$$= 499500$$

Result

Almost half a million different keys may be required.

Large-Scale Example

Suppose a system contains:

$$N = 10000$$

applications.
Then:

$$\frac{10000(9999)}{2}$$

$$= 49,995,000$$

Result

Nearly 50 million pairwise keys may be required.

Solution: Key Distribution Center

A common solution is to use a:

Key Distribution Center (KDC)

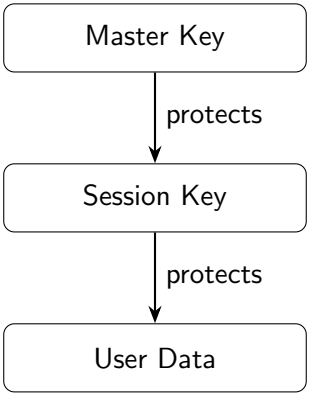
The KDC is a trusted entity responsible for distributing secret keys to communicating users.



Key Hierarchy

A KDC-based system normally uses at least two levels of keys.

1. **Master Key**
2. **Session Key**



Master Key

Master Key

A master key is a long-term key shared between a user and the KDC.

For example:

$$A \leftrightarrow KDC : K_A$$

$$B \leftrightarrow KDC : K_B$$

Each user has a unique master key with the KDC.

Session Key

Session Key

A session key is a temporary key used to protect communication during a particular session.

Example:

$$K_s$$

may be used by A and B during one logical connection.

After the session finishes:

$$K_s \rightarrow \text{Discarded}$$

Master Key vs Session Key

Feature	**Master Key**	**Session Key**
Lifetime	Long-term	Temporary
Shared Between	User and KDC	Two communicating users
Purpose	Protect key distribution	Protect user data
Frequency of Change	Less frequent	Usually frequent
Example	K_A, K_B	K_s

KDC-Based Key Distribution Scenario

Assume:

A wants to communicate with B

and requires a new session key.
The basic entities are:



Step 1: Requesting a Session Key

User A sends:

$$\boxed{ID_A \parallel ID_B \parallel N_1}$$

to the KDC.
Where:

- ID_A: Identity of A
- ID_B: Identity of B
- N_1: Nonce generated by A

What is a Nonce?

Nonce

A nonce is a value intended to be different for each protocol execution.

A nonce may be:

- Random number
- Counter
- Timestamp

A nonce helps prevent:

Replay Attacks

Step 2: KDC Generates the Session Key

The KDC generates:

$$K_s$$

and sends information to A encrypted using A's master key:

$$E(K_A, [K_s \parallel ID_A \parallel ID_B \parallel N_1])$$

The KDC also creates information intended for B:

$$E(K_B, [K_s \parallel ID_A])$$

Step 3: A Sends the Ticket to B

A forwards the information intended for B:

$$\boxed{E(K_B, [K_s \parallel ID_A])}$$

Because it is encrypted using K_B:

- Only B can recover the session key.
- B learns that the communicating party is A.
- B knows that the information originated through the KDC.

Result After Step 3

Now both A and B possess:

$$\boxed{K_s}$$

They can begin encrypted communication:

$$A[\text{Encrypted Data}]K_sB$$

However, additional authentication steps may be used.

Step 4: Authentication Challenge

B generates another nonce:

$$N_2$$

and sends:

$$\boxed{E(K_s, N_2)}$$

to A.

This challenges A to prove possession of the newly established session key.

Step 5: Authentication Response

A applies a transformation:

$$f(N_2)$$

and sends:

$$\boxed{E(K_s, f(N_2))}$$

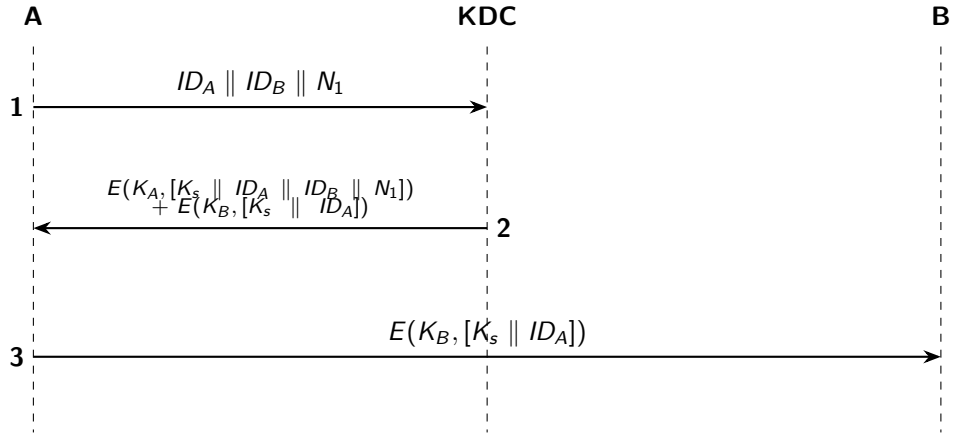
back to B.
Example:

$$f(N_2) = N_2 + 1$$

Purpose

This helps B verify that the received session information is fresh and not simply a replay.

KDC Protocol Flow – Key Distribution

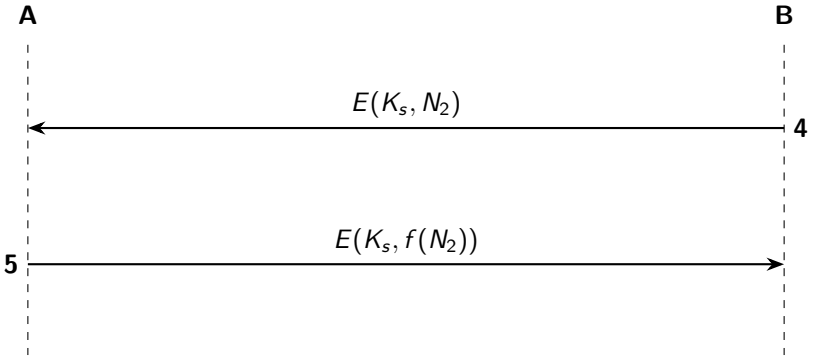


After Step 3

Both A and B securely possess the session key K_s.

KDC Protocol Flow – Authentication

After receiving the session key, B verifies that A is an active participant and that the exchange is not a replay.



Purpose

N_2 acts as a fresh challenge. A's correct response $f(N_2)$ demonstrates possession of K_s and helps prevent replay.

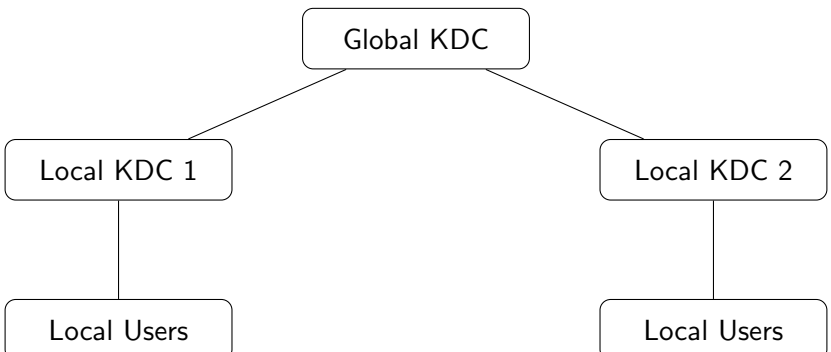
Security Benefits of the KDC Scenario

The protocol provides:

- Secure session-key distribution
- Freshness through nonces
- Replay protection
- Authentication support
- Reduced manual key-distribution burden

Hierarchical Key Control

A single KDC may become impractical in very large networks.
A solution is to create a hierarchy:



Advantages of Hierarchical KDCs

- Local KDCs handle communication inside local domains.
- Global KDCs assist communication across domains.
- Reduces burden on a single centralized KDC.
- Limits damage if one local KDC is compromised.
- Reduces master-key distribution effort.

Session Key Lifetime

How long should a session key be used?

Short Lifetime

- Better security
- Less ciphertext per key
- More frequent distribution

Long Lifetime

- Lower overhead
- Fewer key exchanges
- Greater exposure if compromised

Choosing Session Key Lifetime

For connection-oriented communication:

- One session key may be used for one logical connection.
- A new session may receive a new key.
- Very long connections may periodically change the session key.

For connectionless communication:

- One key per transaction gives stronger security.
- However, this increases overhead.
- A compromise may be to limit use by time or transaction count.

Automatic Key Distribution

A security service can automatically obtain keys from a KDC.
Typical process:

1. Host requests a connection.
2. Security service temporarily buffers the request.
3. Security service obtains a session key from the KDC.
4. KDC delivers the session key.
5. Communication begins using the new session key.

Decentralized Key Control

A system does not necessarily need one centralized KDC.
Two users may establish a session key directly if they already share a master key.
Basic idea:

$$A \leftrightarrow B$$

with shared master key:

$$K_m$$

Decentralized Key Distribution Flow

A possible exchange is:

$$A \rightarrow B : ID_A \parallel N_1$$

$$B \rightarrow A : E(K_m, [K_s \parallel ID_A \parallel ID_B \parallel f(N_1) \parallel N_2])$$

$$A \rightarrow B : E(K_s, f(N_2))$$

This removes dependence on a central KDC for the particular exchange.

Limitation of Decentralized Key Control

If every pair of users must share a master key, then:

$$\frac{N(N-1)}{2}$$

master keys may again be required.

Therefore

Fully decentralized symmetric-key distribution is difficult to scale in large systems.

Controlling Key Usage

Different keys may be assigned different purposes.
Examples include:

- Data-encrypting keys
- PIN-encrypting keys
- File-encrypting keys
- Master keys
- Session keys

A system should prevent a key from being used for an unintended purpose.

Key Tags

One approach associates additional information with the key.
A tag may indicate:

- Whether the key is a master key or session key
- Whether the key can be used for encryption
- Whether the key can be used for decryption

This provides basic control over key usage.

Control Vector

A more flexible method associates a:

Control Vector

with each session key.
The control vector:

- Specifies allowed uses of the key
- Can contain multiple fields
- Can be longer than a simple tag
- Is cryptographically linked to the key

Using Asymmetric Encryption for Secret-Key Distribution

Public-key cryptography can be used to distribute a symmetric session key.
Why?

- Public-key algorithms are computationally expensive.
- Symmetric algorithms are more efficient for bulk data.
- Therefore public-key encryption can protect a small session key.

Public-Key Crypto $\rightarrow$ Distribute Session Key

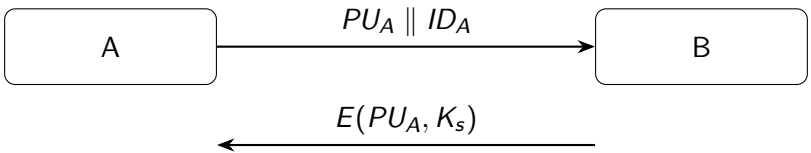
Symmetric Crypto $\rightarrow$ Encrypt Data

Simple Public-Key Session-Key Distribution

A simple method:

1. A generates a temporary public/private key pair.
2. A sends its public key to B.
3. B generates session key K_s.
4. B encrypts K_s using A's public key.
5. A decrypts it using A's private key.
6. A and B use K_s for symmetric encryption.

Simple Secret-Key Distribution Flow



Then:

$$A : D(PR_A, E(PU_A, K_s)) = K_s$$

Problem: Man-in-the-Middle Attack

The simple protocol is vulnerable if an attacker can replace public keys.



The attacker may substitute:

$$PU_E$$

for:

$$PU_A$$

and obtain the session key.

How the MITM Attack Works

1. A sends its public key.
2. E intercepts it.
3. E sends its own public key to B.
4. B encrypts the session key using E's key.
5. E decrypts and learns the session key.
6. E re-encrypts the key and forwards it to A.

Result

A and B may believe their communication is secure while E knows the session key.

Secret-Key Distribution with Authentication

A stronger method uses:

- Public keys
- Nonces
- Authentication
- Encryption

The goal is to provide:

$$\boxed{\text{Confidentiality} + \text{Authentication}}$$

during session-key distribution.

Nonce-Based Authentication

A possible exchange begins with:

$$A \rightarrow B : E(PU_B, [N_1 \parallel ID_A])$$

Then:

$$B \rightarrow A : E(PU_A, [N_1 \parallel N_2])$$

The presence of N_1 demonstrates that B successfully decrypted A's original message.

Completing Authentication

A returns:

$$E(PU_B, N_2)$$

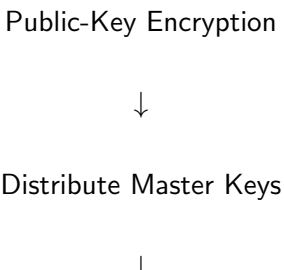
This proves to B that A successfully decrypted the previous response.
After authentication, A can securely send the session key.

Hybrid Key Distribution

A hybrid approach combines:

- Public-key encryption
- KDC
- Symmetric session keys

A possible hierarchy is:



Why Use a Hybrid Scheme?

- Public-key operations are computationally expensive.
- Session keys may change frequently.
- Symmetric encryption is efficient for frequent key handling.
- Public-key encryption can be used less frequently for master-key distribution.

Benefit

Combines the convenience of public-key cryptography with the efficiency of symmetric cryptography.

Distribution of Public Keys

Four general approaches are:

1. Public announcement
2. Publicly available directory
3. Public-key authority
4. Public-key certificates

1. Public Announcement

A user simply publishes or sends their public key.
Example:

$$A: \quad PU_A$$

is publicly distributed.

Advantage

Very simple and convenient.

Problem

An attacker can publish a forged public key pretending to belong to another user.

2. Publicly Available Directory

A trusted organization maintains records such as:

$$\{Name,\ Public\ Key\}$$

Example:

$$\{Alice,\ PU_A\}$$

$$\{Bob,\ PU_B\}$$

Users register their public keys securely with the directory authority.

Weakness of a Public-Key Directory

The directory provides better security than uncontrolled announcements.
However:

- The directory authority must be trusted.
- Records must be protected from modification.
- Compromise of the authority can compromise many users.

3. Public-Key Authority

A public-key authority provides stronger control.
Typical process:

1. A requests B's current public key.
2. Authority returns B's public key.
3. Response is protected using the authority's private key.
4. A verifies it using the authority's public key.

Why Include a Timestamp?

The authority may include a timestamp:

$$T_1$$

in its response.
This helps A determine:

- Whether the response is current.
- Whether an attacker is replaying an old key.
- Whether the public key may have been replaced.

Limitation of a Public-Key Authority

A major problem is:

Online Dependency

A user may need to contact the authority whenever another participant's public key is required.

This can create:

- Communication overhead
- Processing overhead
- Possible bottleneck

4. Public-Key Certificates

A certificate provides a way to distribute a public key without contacting the authority for every communication.
A certificate typically contains:

- Identity of the owner
- Public key of the owner
- Additional information
- Signature of a trusted Certificate Authority

Certificate Authority

Certificate Authority (CA)

A trusted entity that creates and signs public-key certificates.

Example certificate concept:

$$C_A = E(PR_{auth}, [T_1 \parallel ID_A \parallel PU_A])$$

The authority's private key is used to protect the certificate information.

Certificate Verification

A participant receiving a certificate can verify it using the authority's public key:

$$PU_{auth}$$

The participant can then determine:

- Certificate owner's identity
- Certificate owner's public key
- Whether the certificate came from the trusted authority
- Whether the certificate is sufficiently current

Requirements of a Public-Key Certificate

A useful certificate system should allow participants to:

1. Determine the owner's identity and public key.
2. Verify that the certificate was created by the trusted authority.
3. Detect counterfeit certificates.
4. Verify whether the certificate is still current.

X.509 Certificates

A widely accepted format for public-key certificates is:

X.509

X.509 certificates are widely associated with network-security applications such as:

- Transport Layer Security
- IP security
- Secure electronic mail systems

Comparison of Public-Key Distribution Methods

Method	**Advantage**	**Main Problem**
Public Announcement	Very simple	Forgery possible
Public Directory	Centralized records	Authority must be secure
Public-Key Authority	Strong authentication	Authority may become bottleneck
Certificates	Can be exchanged directly	Requires trusted CA

Symmetric vs Asymmetric Key Distribution

Aspect	**Symmetric-Based**	**Asymmetric-Based**
Central Entity	Often KDC	May use CA/authority
Long-term Secret	Master key	Private key
Efficiency	High	Lower
Typical Use	Session-key delivery	Protect session/master keys
Scalability	KDC reduces key count	Public keys simplify distribution

Classroom Numerical

A network contains:

$$N = 500$$

users.
If every user pair requires a unique symmetric master key, calculate the maximum number of keys required.

$$\text{Keys} = \frac{N(N-1)}{2}$$

Classroom Numerical

A network contains:

$$N = 500$$

users.
If every user pair requires a unique symmetric master key, calculate the maximum number of keys required.

$$\text{Keys} = \frac{N(N-1)}{2}$$

$$= \frac{500(499)}{2}$$

Classroom Numerical

A network contains:

$$N = 500$$

users.
If every user pair requires a unique symmetric master key, calculate the maximum number of keys required.

$$\text{Keys} = \frac{N(N-1)}{2}$$

$$= \frac{500(499)}{2}$$

$$= 124750$$

Think and Discuss

Consider a university network with:

- 10,000 students
- 1,000 faculty members
- Hundreds of servers and applications

Would manually distributing symmetric keys be practical?

What architecture would you prefer?

- Pairwise keys?
- KDC?
- Hierarchical KDC?
- Public-key certificates?
- Hybrid model?

Quick Check

Answer briefly:

1. What is the difference between a master key and a session key?
2. Why is a nonce used?
3. What problem does a KDC solve?
4. Why is public-key encryption useful for distributing symmetric keys?
5. What is a man-in-the-middle attack?
6. What is the role of a Certificate Authority?

Descriptive Questions

1. Explain symmetric-key distribution using a KDC.
2. Explain the complete KDC session-key distribution protocol.
3. Discuss hierarchical and decentralized key distribution.
4. Explain symmetric-key distribution using public-key encryption.
5. Explain public announcement, directory, authority, and certificate methods.
6. Explain the purpose of public-key certificates.

References

- William Stallings, *Cryptography and Network Security: Principles and Practice*.
- Chapter on Key Management and Distribution:
  - Symmetric Key Distribution Using Symmetric Encryption
  - Symmetric Key Distribution Using Asymmetric Encryption
  - Distribution of Public Keys
  - Public-Key Certificates

Questions & Discussion

Key management is not only about generating strong keys—
it is also about securely distributing, storing, using, and replacing them.

Your Feedback Matters!

Please scan the QR code and share your feedback about today's lecture.



TCS-392 – Introduction to Cryptography
Lecture 16 – Key Management and Distribution